Exercise 3 Report

6.0 VU Advanced Database Systems (Summer 2026)

Tomas Hobza
Student ID: e12534793
e12534793@student.tuwien.ac.at

June 15, 2026

Contents

1 Exercise 1 – Scalar and Vector Clocks

1.1 a) Scalar clocks

Since the exercise sheet came with a fillable graphic, I believe the answer will be best shown by filling the graphic (see 1) and a short comment on how the result came to be.

The algorithm is quite simple:

1. The sender node increments it's clock upon sending a message.
2. The receiver picks the bigger of the two (it's own clock and the incoming clock state attached to the incoming message), increments it and saves it as it's new clock state.

We can elegantly separate the computation per-process/node, since the only dependencies are: the node's own clock and the incoming clock state attachment.

Process A

1. Initial state: 1
2. Receive m_1: $max(5, 1) + 1 = 6$
3. Send m_3: 7
4. Receive m_2: $max(7, 6) + 1 = 8$
5. Send m_4: **9**

Process B

1. Initial state: 4
2. Send m_1: 5
3. Send m_2: **6**
4. Receive m_4: $max(9, 6) + 1 = 10$
5. Receive m_5: $max(10, 9) + 1 = \mathbf{11}$

Process C

1. Initial state: 3
2. Receive m_3: $max(7, 3) + 1 = \mathbf{8}$
3. Send m_5: 9

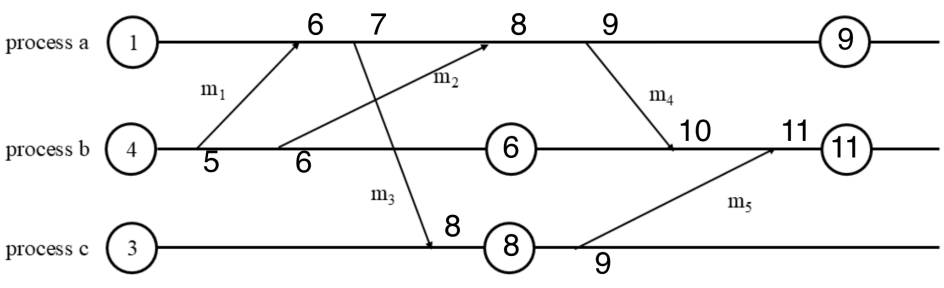

Figure 1: Filled in diagram of communication

1.2 b) Vector clocks

This exercise is again provided with a graphic that can be filled with the correct answers (see 2).

The algorithm is a bit more complex than for scalar clocks:

1. The sender node increments it's own component in the clock vector upon sending a message.
2. The receiver does a component-wise max pick between it's own and the incoming attached clock vectors. Finally, it increments it's own component.

Again, we can separate the computation process-wise. I will write the clock vectors as three-integer tuples in the order: a, b, c.

Process A :

1. Initial state: $(3, 1, 2)$
2. Send m_2: $(4, 1, 2)$
3. Receive m_3: $(max(4, 5) + 1, max(1, 5), max(2, 6)) = (6, 5, 6)$
4. Receive m_4: $(max(2, 6) + 1, max(5, 6), max(3, 6)) =$ **(7,6,6)**

In the spirit of environmental consciousness, I will omit the component-wise max picking from now on to save the electricity and my keyboard's lifespan.

Process B :

1. Initial state: $(2, 4, 3)$
2. Send m_1: **(2,5,3)**
3. Send m_4: $(2, 6, 3)$
4. Receive m_5: **(5,7,7)**

Process C :

1. Initial state: $(5, 2, 1)$
2. Receive m_1: $(5, 5, 4)$
3. Receive m_2: $(5, 5, 5)$
4. Send m_3: **(5,5,6)**
5. Send m_5: $(5, 5, 7)$

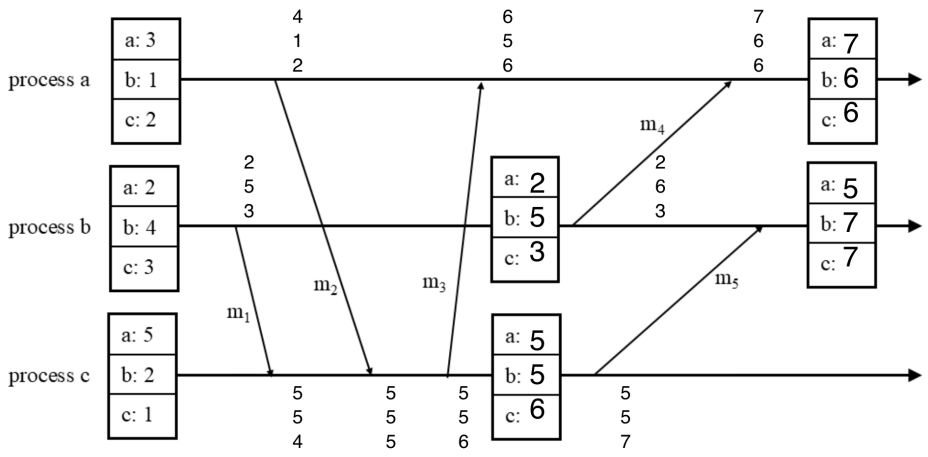

Figure 2: Filled graphic with vector clocks

2 Exercise 2 – Denormalization

In this exercise, we have four tables – `room`, `booking`, `occupancy`, `guest` – in a traditional relational database and we want to translate the data into a document store. The additional constraints for the data model are:

- Listing the names of guests who stayed at the hotel in a given year has to be a fast read operation.
- Updating the `booking` relation has to also be fast.

Right away, we can see that the pair of constraints is in tension with each other. One may denormalize the guest names for the respective years, but this information is dependent on the bookings – denormalizing would mean sacrificing the speed of updates of bookings.

It is apparent that the optimal solution is going to be a compromise. My reasoning on how to find a satisfactory solution is the following:

In the original relational model, updates on the `booking` relation are very fast, but fetching the list of guest names that stayed in the hotel in a given year requires joining the `guest` relation onto the `booking` relation followed by a date filter on the `booking.check_in` attribute.

To prevent having to execute this join, I propose denormalizing the `guest` relation onto the `booking` relation. In addition, I propose extracting the check-in year into an attribute of its own and creating an index on that new year attribute. The query becomes a single indexed filter `find(year: Y)` projecting `guest_name`, with no join, and distinct applied since a guest can book more than once in the same year. However, it is important to state that this assumes that a booking is always fully contained within one year. If that assumption didn't hold (the example entries don't show such a case), one would have to update the data model to allow for this $1 : 1...2$ cardinality.

The proposed `booking` collection documents will look like the following:

```
{
    "_id": 1,
    "guest_id": 3,
    "guest_name": "Bob",
    "room": 1,
    "nights": 3,
    "check_in": "05-10-2025",
```

```
    "year": 2025
}
```

It is important to note that this tradeoff makes guest name changes very expensive (each booking has to be updated) – in real scenarios this may be a benefit due to audit trails: the booking is archived with the name the guest had while making said booking.

We still need the simple `guest` relation to remain in the model for the guests that do exist in the system, but have not yet made a booking. For that purpose, I included the `guest` collection:

```
{
    "_id": 3,
    "name": "Bob"
}
```

Lastly, the remaining two relations, `room` and `occupancy`. Since the assignment states that updates to these relations are rare, room and occupancy can be merged using a standard embedding approach. The resulting `room` collection documents look like the following:

```
{
    "_id": 1,
    "type": "Single",
    "price": 89,
    "occupancy": [
        {"year": 2024, "available_nights": 280},
        {"year": 2025, "available_nights": 310}
    ]
}
```

3 Exercise 3 – Graph Databases

3.1 Part 1

Task I

```
MATCH
    (l:Language)<-[:SPEAKS_LANGUAGE]-(p:Person)-[:RECEIVED_AWARD]->(a:Award)
RETURN DISTINCT l;
```

Figure 3: Output for Exercise 3, Part 1, Task I

Task II

```
MATCH
    (c:Country)<-[:LOCATED_IN]-(u:University)<-[:STUDIED_AT]-(p:Person)
RETURN DISTINCT c;
```

Figure 4: Output for Exercise 3, Part 1, Task II

Task III

```
MATCH
    (c:Country)<-[:LOCATED_IN]-(u:University)<-[:STUDIED_AT]-(p:Person)
RETURN DISTINCT c, COUNT(DISTINCT p) as people_count;
```

Figure 5: Output for Exercise 3, Part 1, Task III

Task IV Here, it is important to note that this task assignment can be understood as an extension of the previous task by introducing a new column that holds the count only of the persons meeting the criteria. That would require an optional match and the query complexity would increase. I understood and implemented it to find a subset of the previous task for which all the criteria are met.

```
MATCH
    (c:Country)<-[:LOCATED_IN]-(u:University)<-[:STUDIED_AT]-(p:Person)
```

```
    -[:SPEAKS_LANGUAGE]->(l:Language)
WITH c, p, count(DISTINCT l) as lang_count
WHERE lang_count >= 2
RETURN DISTINCT c, COUNT(DISTINCT p) as people_count
ORDER BY people_count DESC;
```

Figure 6: Output for Exercise 3, Part 1, Task IV

3.2 Part 2

Task I Note: The query does not specify the type of the node that is employed at a university. This is because the task assignment states to count the number of incoming `EMPLOYED_AT` edges and a university might employ a service dog that still is an employee, but not a Person.

```
MATCH
    (u:University)<-[:EMPLOYED_AT]-(p)
RETURN u.name, COUNT(p) as p_count
ORDER BY p_count DESC
LIMIT 10;
```

Figure 7: Output for Exercise 3, Part 2, Task I

Task II

```
MATCH
    (p:Person)-[:BORN_IN]->()-[:LOCATED_IN]->(r:Region)
RETURN r.name, COUNT(DISTINCT p) as p_count
ORDER BY p_count DESC
LIMIT 10;
```

Figure 8: Output for Exercise 3, Part 2, Task II

Task III

```
MATCH
    (c:Country)<-[:LOCATED_IN]-(:University)<-[:EMPLOYED_AT]-
        (p:Person)-[:RECEIVED_AWARD]->(:Award {name: "Turing Award"}),
    (p)-[:RECEIVED_AWARD]->(:Award {name: "ACM Fellow"})
RETURN c.name, COUNT(DISTINCT p) as p_count
ORDER BY p_count DESC
LIMIT 3;
```

Figure 9: Output for Exercise 3, Part 2, Task III

3.3 Part 3

Note: I assume that we accept the case when the two distinct persons received the same award. If that shall not be accepted, a secondary condition to filter that case out is to be added into the `WHERE` part.

```
MATCH
    (p1:Person)-[:BORN_IN]->(p:Place)<-[:BORN_IN]-(p2:Person),
```

```
    (p1:Person)-[:STUDIED_AT]->(u:University)<-[:STUDIED_AT]-(p2:Person),
    (p1:Person)-[:RECEIVED_AWARD]->(a1:Award),
    (p2:Person)-[:RECEIVED_AWARD]->(a2:Award)
WHERE p1 <> p2
RETURN p1, p2, u, p, a1, a2 LIMIT 1;
```

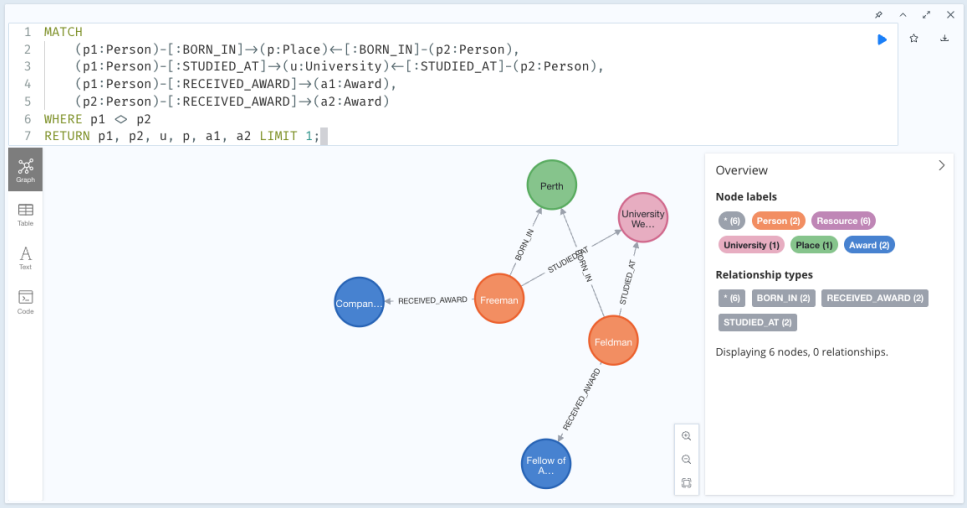

Figure 10: Output for Exercise 3, Part 3

3.4 Part 4

First, the `MERGE` query:

```
// 1. Names
MATCH (p:Person)-[:RECEIVED_AWARD]->(:Award)
WITH DISTINCT p
WHERE p.last_name IS NOT NULL
MERGE (n:Name {name: p.last_name})
MERGE (p)-[:HAS_NAME]->(n);
// 2. Dates of birth
MATCH (p:Person)-[:RECEIVED_AWARD]->(:Award)
WITH DISTINCT p
WHERE p.date_of_birth IS NOT NULL
MERGE (dob:DateOfBirth {date_of_birth: p.date_of_birth})
MERGE (p)-[:HAS_DOB]->(dob);
```

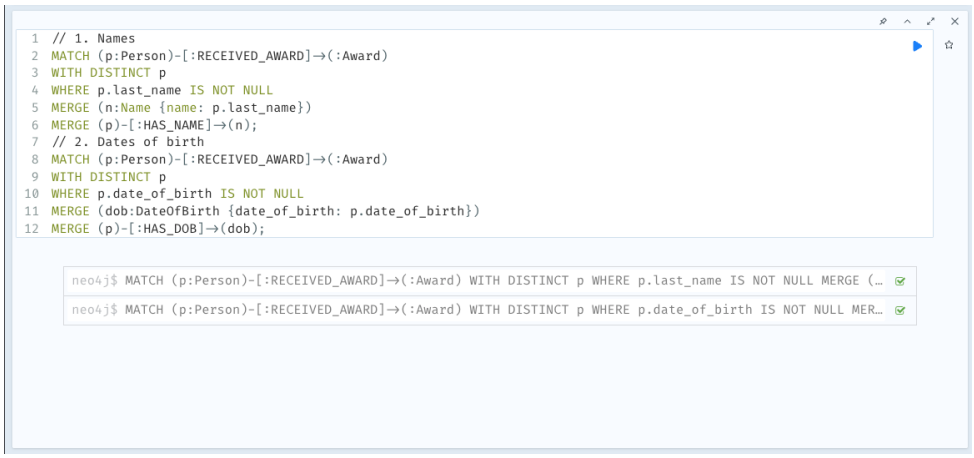

Figure 11: Output for Exercise 3, Part 4, Merge Query

I have split the merge query so it is more clean. The main reason why it has to be split is that we cannot just join the two statements, since if a `Person` node is missing a last name, we should not create just the `Name` node and relation while not impacting the date of birth flow.

Now for the `MATCH` query:

```
MATCH (p:Person)-[:HAS_NAME]->(n:Name)
WITH count(DISTINCT p) AS p_cnt, n
ORDER BY p_cnt DESC
LIMIT 5
WITH collect(n) AS top_names
MATCH (p:Person)-[:HAS_NAME]->(n:Name)
WHERE n IN top_names
RETURN p, n
```

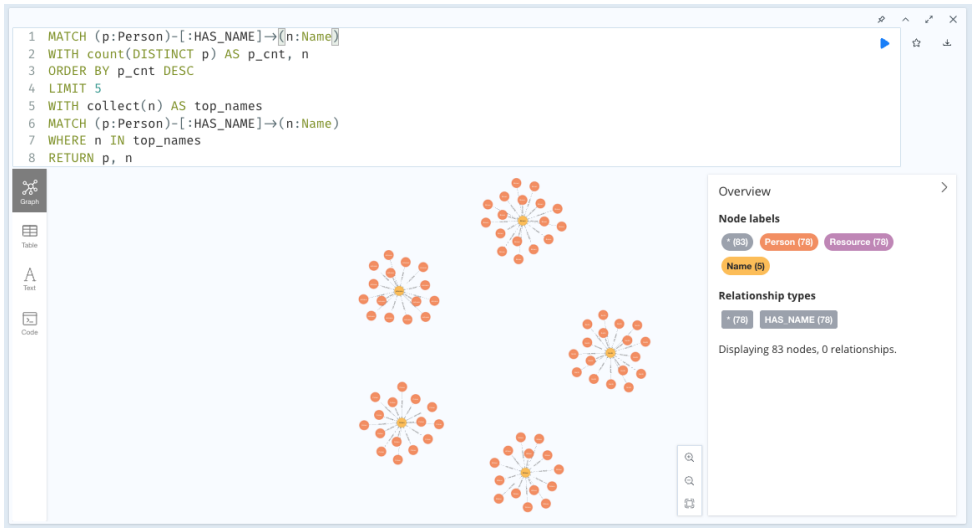

Figure 12: Output for Exercise 3, Part 4, Match Query

3.5 Part 5

First, the `MERGE` query:

```
MATCH
```

```
    (p1:Person)-[:STUDIED_AT]->(u:University),
    (p1:Person)-[:STUDIED_AT]->(v:University),
    (p2:Person)-[:EMPLOYED_AT]->(u:University),
    (p2:Person)-[:EMPLOYED_AT]->(v:University)
WHERE v <> u AND p1 <> p2
MERGE (u)-[:CONNECTED]-(v)
```

Figure 13: Output for Exercise 3, Part 5, Merge Query

Now, the match query:

```
MATCH
    (a:University {name: "TU Wien"}),
    (b:University)-[:LOCATED_IN]->(:Country {name: "Germany"})
MATCH path = shortestPath((a)-[:CONNECTED*]-(b))
RETURN path
```

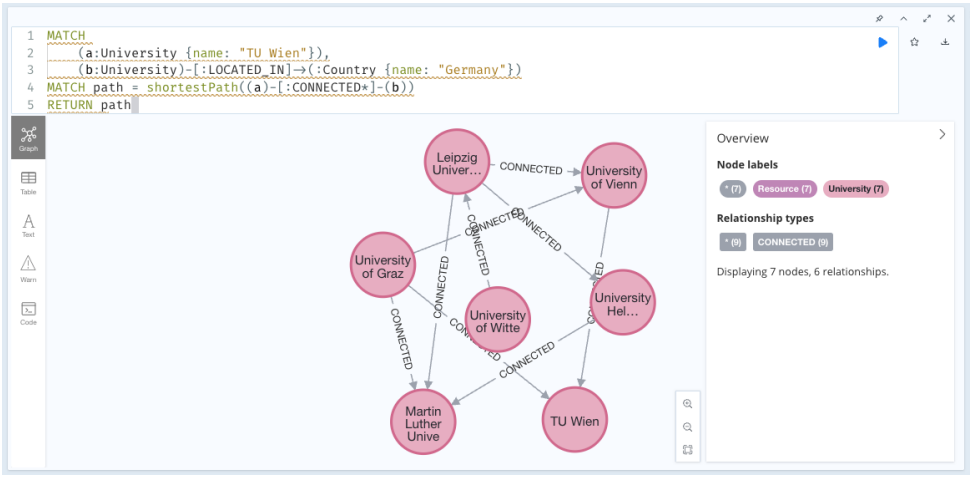

Figure 14: Output for Exercise 3, Part 5, Match Query

4 Exercise 4 – Late Materialization

The intermediate results for each step of the query plan are shown below. Columns `posL`/`posR` are positions in R/S; for `join_input_S` each row is an S_a value with its position.

inter1

pos
2
4
5

inter2

pos	Rb
2	34
4	56
5	22

join_input_S

Sa	pos
34	1
22	2
17	3

join_res

posL	posR
2	1
5	2

inter3

pos
2
5

inter4

pos	Ra
2	16
5	5

result

result
21